CS781 Capstone Project: In-House Legal Intelligence System

Manish Kumar
251330604
{manishkmr25}@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

Imagine a law firm that wants AI help with its case files but cannot let a single page leave the building. That was our brief. We built everything ourselves: four small legal language models (82–89M parameters) trained from scratch on Indian court and legislative text, and six tools on top of them that find named entities, label the role of each sentence, predict case outcomes, identify statutes, write summaries and retrieve prior cases. Along the way we found that the starter data leaked validation text and was full of duplicates and OCR noise; cleaning it cut A4's perplexity from 13.3 to 11.3 at the same budget. The biggest gains came from respecting how legal documents are built: reading the end of a judgment for its outcome (0.754 to 0.785), and matching each citation on its own when retrieving precedents (0.32 to 0.43).

1 Competition Result

Codabench username: m_251330604 **Roll number:** 251330604
GitHub repository: https://github.com/manishkr21/capstone_llm

Here is where we stood on the leaderboard when we froze the report.

Table 1: Official phase scores and ranks. Fill every cell. State the date on which ranks were recorded.

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	BERTScore F1: 0.8286	95	6
2	Legal NER	strict macro-F1: 0.4970	80	6
3	Rhetorical role prediction	macro-F1: 0.5750	15	51
4	Court judgment prediction	macro-F1: 0.8121	13	30
5	Legal statute identification	macro-F1: 0.4624	6	50
6	Abstractive summarization	BERTScore F1: 0.8558	4	32
7	Prior case retrieval	micro-F1@7: 0.4950	_____	21

Total valid Codabench submissions across all phases: 196 **Date ranks were recorded:** 6 November 2026

We cleared the minimum of four distinct submissions in every phase (six or more each); the six best of each phase, and what produced them, are listed in Appendix A.

2 Introduction and Problem Description

Law firms sit on mountains of text, and most of it is confidential. A firm that wants language technology therefore cannot simply call a commercial API, and it cannot fine-tune someone else's

Table 2: Tasks (inputs, outputs and primary metrics as scored on Codabench).

Ph.	Task	Input → output	Primary metric
1	Legal LM	512-word legal prompt → 128-word continuation	BERTScore F1
2	L-NER	judgment text → character spans with one of 12 entity types	strict macro-F1
3	Rhetorical roles	sentences of a judgment → one of 13 roles each (Corporate Law, IT)	macro-F1
4	Judgment prediction	judgment (outcome removed) → allowed (1) / dismissed (0)	macro-F1
5	Statute identification	case text (statute names masked) → set of statute ids (of 100)	macro-F1
6	Summarization	judgment → abstractive summary	BERTScore F1
7	Prior case retrieval	query judgment + candidate pool → 7 ranked case ids	micro-F1@7

Table 3: Data per phase as used (after de-duplication and leak removal). Lengths are medians.

Ph.	Split sizes	Statistics that informed the design
1	1.006B train tokens (cleaned), 4.10M val tokens	3.48M documents read; 106,717 exact and 204,246 near duplicates removed; 90,856 documents with OCR word stutter repaired; 764 validation documents dropped as leaks
2	70 train docs (10 held out), 35 test	8,393 entities in 12 types; median 4,488 words per document
3	90 labelled docs (45 CL, 45 IT), 10 test	labels persist: next sentence has the same role with probability 0.94 (CL) / 0.75 (IT); PrecedentOverruled has 3 training sentences
4	31,677 train, 1,988 clean val, 1,517 test	58.6% of training labels are 0, validation and test balanced; median 2,469 tokens (train), 3,771 (val)
5	42,315 train, 10,024 val, 13,019 test	100 statutes, 3.71 per document; frequency 270–11,577 documents; statute names masked as `<SECTION>`/`<ACT>`
6	6,703 train, 300 val (ours), 100 test	documents 3,123 words, summaries 640 words (compression 4.6); 75% of references exceed the ≈400 words that BERTScore compares
7	827 / 118 / 237 queries; pools 4,320 / 1,023 / 1,727	6.4–8.2 gold cases per query (range 0–92); citations masked as `<CITATION>`; a perfect 7-item list scores only 0.618 on dev

foundation model whose data and licence it does not control. The capstone turns this into a concrete challenge: train your own compact language model (50–300M parameters, no pretrained weights, only the official `Exploration-Lab/CS-781-Capstone` data), then teach it six real legal tasks taken from the IL-TUR benchmark [1]. Table 2 lists them.

Three things made this harder than it sounds. Legal documents are *long*: a typical judgment runs to thousands of words (median 2,469 tokens in Phase 4, 4,508 words in Phase 7), while our models can only look at 1,024 tokens at a time. The *metrics are unforgiving*: an entity counts only if every character matches, rare classes weigh as much as common ones, and a summary is compared with a single reference. And we had *one GPU*, shared with other people's jobs.

So we kept asking the same few questions. Does cleaning the data matter more than the architecture? How well do our small models carry over to each task? Can cheap adaptation methods match full fine-tuning? How do you feed a 20-page judgment to a model that reads two pages? And how much do decoding and threshold choices move the score? We built and submitted every phase end to end. We did not chain the phases together, train a separate tokenizer per task, or have lawyers judge the outputs.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

We used only the official dataset. Table 3 shows what each phase looked like after we cleaned it, and the numbers that ended up shaping our design.

Surprises in the data. Before training anything, we checked the files, and they were not always what they seemed. The Phase-1 starter split had validation documents copied into training, so

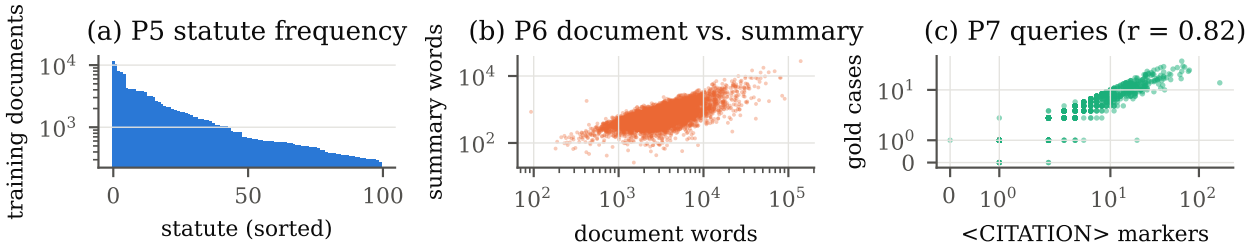


Figure 1: (a) Phase-5 statutes per training document count, sorted (log scale). (b) Phase-6 document length against summary length. (c) Phase-7 training queries: number of `<CITATION>` markers against number of gold cases.

we built a fresh validation set that shares no text with either corpus. In Phase 4 the test file still contained its labels, and every test document also appeared, labelled, in the development file; we never read those labels and chose models only on the 1,988 development documents not in the test set. In Phase 5 we removed development texts that also sat in the test file (96) or in training (118), and merged 435 duplicates. Phase 7 had the most interesting quirk: every query appears in its own candidate pool, and almost every other case in a pool (99.9% on train, 100% on dev) is the correct answer for *some* query. We used that later.

3.2 Preprocessing decisions

Every model shares one byte-level BPE tokenizer [2] with 32,000 tokens, trained on legal text, so Devanagari names, section symbols and OCR debris never turn into "unknown". Cleaning the Phase-1 corpus meant normalising characters, repairing OCR stutter ("the the the court"), dropping short or garbled documents, and removing exact and near-duplicates. NER needed special care because a single character off means a miss: we split runs of punctuation before tokenizing and kept a map from every token back to its characters, so spans survive the round trip (99.45% exactly, up from 98.66%). For long documents each phase found its own trick: read eight 512-token chunks (the *last* eight for outcomes), condense a judgment to its opening plus its most central sentences for summaries, or cut out the ±60 words around each citation for retrieval.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

We built four quite different models and trained them identically, so that any difference comes from the architecture alone (Table 4). All learn to predict the next word on 1,024-token windows; the encoder–decoder learns to continue a random-length prefix, and the prefix-LM reads part of its input in both directions. Our first models (R0) used the starter sample of about 200M tokens. After the data audit we retrained them on the cleaned corpus at the same budget (R1), then kept going to 1B tokens (R2). For the submission, text is sampled with temperature 0.8, top-k 50, top-p 0.95 and a mild repetition penalty [3], and the model is not allowed to stop before 128 words.

4.2 Transfer protocol (Phases 2–7)

From here on, every task model starts from one of our Phase-1 checkpoints and learns only from that task's data (Figure 2). A small wrapper turns each language model into an encoder. We learned along the way that A1 and A4 work best when they keep their original left-to-right attention,

Table 4: The four Phase-1 architectures (vocabulary 32k, context 1,024).

ID	Design	Params
A1	Decoder-only [4]; learned positions, pre-LayerNorm, GELU; 8 layers, $d = 768$, 12 heads, $d_{\text{ff}} = 3072$	82.1M
A2	Encoder–decoder [5], 6+6 layers, sinusoidal positions, $d = 640$, 10 heads	89.4M
A3	Prefix-LM: bidirectional prompt, causal continuation, ALiBi [6]; 8 layers, $d = 768$	81.3M
A4	RoPE [7], RMSNorm, SwiGLU, grouped-query attention [8] (12 q / 4 kv heads), 256-token sliding window [9] with every third layer global; 10 layers, $d = 768$	87.5M

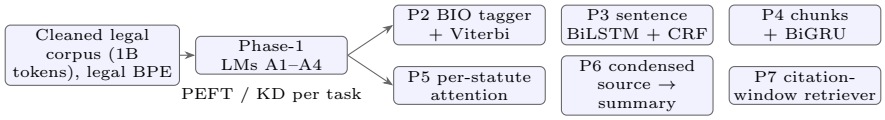


Figure 2: One pre-trained backbone family, six separately fine-tuned descendants.

that A3 should read its input in both directions, and that A2's encoder had collapsed during pre-training, so we use its decoder instead. We wrote each adaptation method once and reused it everywhere: full and partial fine-tuning, gradual unfreezing [10], BitFit [11], LoRA [12], LoRMA [13], QLoRA [14], AdaLoRA [15], Houlsby adapters [16], IA^3 [17], prefix [18] and prompt tuning [19], and knowledge distillation [20] into smaller students. Every phase tried at least six of them, across all four architectures.

4.3 Phase 2 — Legal NER

We turned entity spans into B/I/O tags on tokens, let each backbone tag overlapping 512-token windows (each token trusts the window where it sits most centrally), and decoded with a Viterbi pass that refuses impossible tag sequences, before mapping tags back to exact characters. Six experiments covered all four architectures.

4.4 Phase 3 — Rhetorical role prediction

A sentence's role depends on its neighbours, and roles come in long runs, so we never classify a sentence alone. Neighbouring sentences are encoded together, each sentence is pooled into one vector, a BiLSTM reads the whole document, and a CRF [21] picks the most plausible sequence of roles. Corporate Law and IT share one model with a domain flag; rare roles get extra weight through logit adjustment; and with only 90 labelled documents, everything is judged by five-fold cross-validation.

4.5 Phase 4 — Court judgment prediction

A judgment is cut into 512-token chunks; the model reads eight of them, summarises each, and lets a BiGRU with attention [22] decide which chunks matter. Training data leans 59/41 towards dismissals while the test set is balanced, so we weighted the loss and tuned the decision threshold on validation. The metric is macro-F1 rather than accuracy, so that "allowed" and "dismissed" count equally.

4.6 Phase 5 — Legal statute identification

Statutes are usually discussed in one part of a judgment, so we gave each of the 100 statutes its own attention query: it looks for the chunks that talk about it. Rare statutes get larger loss weights ($\sqrt{n_{\text{neg}}/n_{\text{pos}}}$, capped at 30) and their own decision thresholds. Later we trained 6-block

Table 5: Best Codabench submission per phase, its checkpoint(s) and local held-out score.

Ph.	Submission	Checkpoint(s)	Local	Codabench
1	A1 R0	`runs/A1_gpt/best1.pt` (starter corpus, 200M tokens)	0.829	0.8286
2	E2	A4 LoRMA on `runs/A4_efficient/best.pt`, BIO Viterbi	0.486	0.4970
3	T14	A1 LoRA, causal, 1,024-token windows, 5 fold models	0.501	0.5750
4	STACK_with_fusion	logistic-regression stacker over 26 runs (Appendix A)	0.823	0.8121
5	STACK11_full	MLP + LightGBM stackers, 28 Phase-1 + 6 lexical models	0.460	0.4624
6	S3 beam 4	A4 full fine-tuning, 4,096-token source, 4 beams	0.852	0.8558
7	STK4	LightGBM re-ranker + mean of 4 bi-encoders	0.491	0.4950

students to imitate ensembles of our best runs, and finally combined all runs with MLP and per-statute LightGBM [23] stackers, optionally adding simple TF-IDF models (a hybrid that uses no pretrained weights).

4.7 Phase 6 — Abstractive summarization

Here the model reads [a short prompt, the condensed judgment, "`Summary:`"] and learns to write the summary; A2 reads four chunks separately and fuses them in its decoder [24]. Since a summary can only mention what the model has read, we kept stretching the input: stretching A1's position table [25] let sources grow from 1,280 to 8,192 tokens, each step starting from the previous model. Decoding is greedy with a repetition penalty and no repeated 3-grams. As a final step we can pick, for each document, the candidate summary that agrees most with the others (minimum-Bayes-risk selection [26]).

4.8 Phase 7 — Prior case retrieval

Each `<CITATION>` marker in a query hides one cited case, and the words around it describe that case. So instead of matching the whole judgment, we search with each citation's neighbourhood separately using BM25 [27], push down cases decided after the query, and use the pool quirk above: since every case in the pool is cited by *someone*, we also ask which query each case fits best. On top sits a neural retriever built on a Phase-1 model, trained with InfoNCE [28] against hard negatives from BM25, and a LightGBM ranker that re-orders the top 100 candidates using both.

4.9 Final submitted system

5 Experiments and Implementation Details

5.1 Compute and training protocol

Everything ran on one NVIDIA H200 that we shared with other people's jobs, so we could rarely run more than two to four experiments at once. The longest Phase-1 run took about an hour per architecture for 803M tokens; task runs took anywhere from 2 minutes to 3.5 hours. All of them used AdamW with warm-up and cosine decay, gradient clipping and bf16, saved their best state, and could resume after a crash.

5.2 Local evaluation

Because Codabench only shows a score after you upload, we needed honest local scores first. Each phase got its own held-out set built from public data: a leak-free Phase-1 validation set, 10 held-out NER documents (double-checked on three splits), five-fold cross-validation in Phase 3, the

Table 6: Hyperparameters of the final model of each phase (P5–P7: the best single Phase-1 model). *R2, after 3,000 steps of R1.

Hyperparameter	P1 (A4)	P2 (A4 LoRMA)	P3 (A4 LoRMA)	P4 (A4 partial)	P5 (A4 KD)	P6 (A1 full)	P7 (A1 full)
Parameters (M)	87.5	87.5 (2.2 trained)	87.5	88.0 (25.7 tr.)	63.4	85.0	82.3
Context length	1,024	512 windows	512 windows	8×512	8×512	4,096 + 768	256 / 512
Peak LR	6×10^{-4}	10^{-3}	10^{-3}	10^{-4}	5×10^{-5}	5×10^{-5}	3×10^{-5}
Batch × accum.	8 × 8 seq.	16 windows	4 chunks	2 × 8 docs	4 × 4 docs	1 × 16	8 queries
Epochs / steps	12,260 steps*	≤15 (patience 5)	20 (patience 6)	2	8	3 (+6 at 2,560)	10 (best 5)
Dropout	0	0.1	0.1	0.1	0.1	0.1	0
Decoding / threshold	T 0.8, top-p 0.95	BIO Viterbi	CRF Viterbi	0.39	per statute	greedy, pen. 1.15	top 7

Table 7: One-factor comparisons on the local held-out split of each phase. †Three changes at once: causal mask, 20 epochs, logit adjustment $\tau = 0.5$. OOF = out-of-fold, CF = cross-fitted.

Ph.	Factor changed (everything else fixed)	Model, metric	Before	After
1	Corpus cleaning, same 200M tokens	A4, fair-val perplexity	13.32	11.25
1	Token budget 200M → 1B	A4, fair-val perplexity	11.25	8.26
2	Argmax → BIO Viterbi decoding	A4 LoRMA, macro-F1	0.373	0.486
2	Original → cleaned 1B backbone	A4 LoRMA, macro-F1 (3 splits)	0.488	0.467
3	Sentence alone → context window	A4 LoRMA, OOF macro-F1	0.361	0.455
3	Tuning round†	A4 LoRMA, OOF macro-F1	0.455	0.500
4	First 8 → last 8 chunks	A4 partial, macro-F1	0.754	0.785
5	3 → 5 epochs	A4 partial, CF macro-F1	0.370	0.381
5	Spread → statute-placeholder chunks	A4 partial, CF macro-F1	0.370	0.371
5	KD teacher 5 → 8 models	6-block A4 student, CF macro-F1	0.404	0.402
6	Repetition penalty + no repeated 3-grams	9 first-round runs, BERTScore	+0.003 to +0.008	
6	Source 1,280 → 2,560 tokens	A1 full, BERTScore F1	0.8505	0.8520
6	Spread → salient condensation	A1 LoRA, BERTScore F1	0.8452	0.8493
7	Whole query → per-citation BM25	no learning, micro-F1@7	0.324	0.432
7	+ case-wise pool normalisation	BM25 + date penalty, micro-F1@7	0.453	0.469
7	No teacher → distillation from E2	4-block A1, neural micro-F1@7	0.354	0.369

development documents not in the test set in Phase 4, cross-fitted thresholds in Phase 5, 300 held-out documents in Phase 6, and the official dev split in Phase 7. A script checked every zip before upload, catching missing ids, wrong counts and duplicate answers. Local and leaderboard scores do not always agree, partly because some local sets are small and partly because we score summaries with roberta-large while the organisers use a "legal BERTScore".

5.3 Ablations and controlled comparisons

To know what actually helped, we changed one thing at a time. Each row of Table 7 compares two runs that differ only in the factor named.

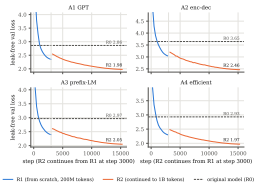
6 Results

Table 8 puts our local scores next to the leaderboard, and Figure 3 and Figure 4 show how training went. Averages hide a lot: two Phase-3 roles have almost no training examples (3 sentences for PrecedentOverruled, 2 documents for Dissent), and in Phase 5 the rarest fifth of statutes averages 0.32 F1 against 0.56 for the most common.

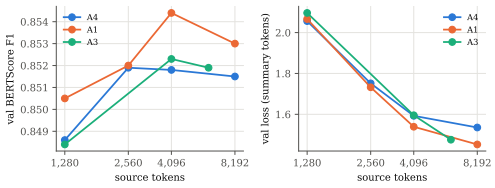
Which system won, and why. The winners differ by phase, but the reasons repeat. In **Phase 1**, A1, A3 and A4 finish within 0.0006 of each other on Codabench, while A2 trails (0.8184): with the same data and budget, the decoder-style models are interchangeable, and the encoder–decoder pays for splitting its capacity between two stacks (its encoder output later turned out to be nearly constant). In **Phase 2**, A4 with LoRMA won with only 2.2M trained parameters. With 70 training documents, updating a small multiplicative correction preserves what pre-training learned better

Table 8: Main variants: local held-out score next to the Codabench test score (– = not uploaded). Secondary (local): P1 ROUGE-L/BLEU-4, P2 and P4 precision/recall, P5 micro-F1, P6 ROUGE-1/ROUGE-L, P7 precision/recall.

Ph.	Variant	Detail	Local	Secondary	Codabench
1	A1 R0	starter corpus, 200M tokens	0.829	0.192 / 0.058	0.8286
1	A4 R0	starter corpus, 200M tokens	0.827	0.186 / 0.050	0.8280
1	A4 R2	cleaned, 1B tokens	0.836	0.213 / 0.093	–
2	E2 A4 LoRMA	2.2M trained parameters	0.486	0.517 / 0.478	0.4970
2	E1 A3 full	full fine-tuning	0.427	0.447 / 0.441	0.4815
2	E6 KD student	4-block A3, 52.9M parameters	0.344	0.372 / 0.344	0.3799
3	T7 A4 LoRMA	tuned, out-of-fold	0.500	–	0.5586
3	T14 A1 LoRA	tuned, 1,024-token windows	0.501	–	0.5750
3	C7 ensemble	8 taggers, out-of-fold	0.521	–	0.5707
4	E6 A4 partial	best single model	0.785	0.787 / 0.786	0.7633
4	ENS_top5	mean of 5 runs	0.797	0.800 / 0.798	0.7930
4	STACK_with_fusion	stacker, nested estimate	0.823	–	0.8121
5	STACK6	stacker, Phase-1 runs only	0.423	0.520	0.4231
5	STACK11	hybrid stacker (+ TF-IDF)	0.460	0.558	0.4624
6	F1 A4 full	1,280-token source, first decode	0.843	0.477 / 0.251	0.8483
6	S7 A1 full	4,096-token source, greedy	0.854	0.513 / 0.273	0.8508
6	S3 A4 full	4,096-token source, beam 4 (100 docs)	0.852	0.525 / 0.275	0.8558
7	STK0	lexical re-ranker only	0.477	0.519 / 0.441	0.4725
7	E2 A1 full	bi-encoder fused with BM25	0.470	–	0.4643
7	STK4	re-ranker + 4 bi-encoders	0.491	0.535 / 0.454	0.4950

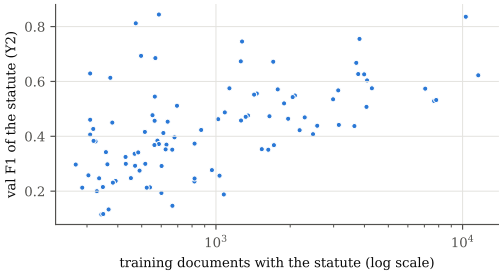


(a) Phase 1: validation loss against training tokens.

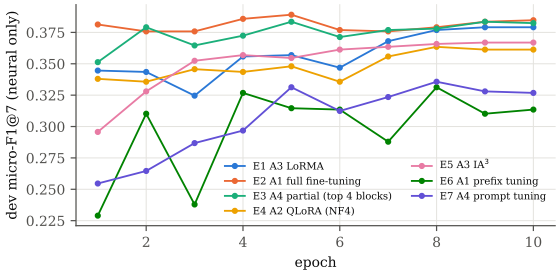


(b) Phase 6: BERTScore and loss against source length.

Figure 3: Learning curves and the source-length experiment.



(a) Phase 5: per-statute F1 (best single model) against training frequency.



(b) Phase 7: dev micro-F1@7 of the bi-encoders after each epoch.

Figure 4: Per-class breakdown (Phase 5) and task metric against epochs (Phase 7).

than rewriting all weights, and the constrained decoder mattered even more than the backbone (argmax 0.373 vs. Viterbi 0.486). In **Phase 3**, the winner (A1 LoRA with 1,024-token windows) is the variant that sees the most neighbouring sentences; adding context was the single largest change we measured (0.361 to 0.455), because roles are defined by where a sentence sits in the argument. In **Phase 4**, the stacker won because its members read different parts of the judgment (last, first, spread chunks) and therefore make different mistakes, and the end of the judgment, where the outcome is argued, carries most of the signal (0.785 vs. 0.754). In **Phase 5**, the hybrid stack beat the Phase-1-only stack (0.462 vs. 0.423 on Codabench): statute names are masked, but TF-IDF models still pick up exact phrases that accompany a statute (offence wording, procedural terms) across the whole document, which eight 512-token chunks cannot all cover, so the two families err

on different documents. In **Phase 6**, longer sources won (1,280 to 2,560 tokens: +0.0015), since a summary cannot mention what the model never read, and beam search helped more on Codabench (0.8558 vs. 0.8520 greedy) than locally. In **Phase 7**, most of the score comes from matching each citation separately and from normalising across the pool (0.324 to 0.483 with no learning); the Phase-1 retrievers add about 0.01–0.02 on top by recognising paraphrased legal reasoning that word overlap misses.

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

Every phase fails in a way that tells us something about legal text. In **NER**, the problem is mostly silence rather than confusion: 37.9% of gold entities are never predicted, against 8.0% with a wrong boundary and 5.6% with a wrong type. Strict span matching turns any boundary slip into a miss, and the commonest type error, appellant tagged as respondent (35 cases), is one that a 512-token window cannot resolve, because who is the appellant is settled pages earlier. In **rhetorical roles**, the model mixes up Ratio and PrecedentRelied (548 and 539 sentences) and Fact and Ratio (499). These sentences use the same vocabulary; what differs is *whose* reasoning it is, a discourse cue the sentence encoder only partly sees. Roles with three training sentences are simply never learned. In **judgment prediction**, errors are fairly balanced (176 false positives, 250 false negatives), and reading the last chunks instead of the first (0.785 vs. 0.754) supports the idea that the outcome is argued near the end. In **statute identification**, the most common error is telling: a missed statute is replaced by the most frequent one, so when evidence is weak the model falls back on its prior, and rare statutes suffer (mean F1 0.32 for the rarest fifth, 0.56 for the most frequent). In **summarization**, scores fall as judgments get longer (correlation -0.40): condensation has to drop more, and whatever it drops cannot appear in the summary. In **retrieval**, 8 of 118 dev queries get no correct case, and queries citing only one to three cases are capped by the fixed list of seven (precision 0.21).

7.2 What the metrics do not measure

None of these numbers checks legal correctness. A summary can score a high BERTScore while stating the wrong holding, because embedding similarity rewards shared vocabulary, and only its first $\approx$400 words are compared at all. Strict span-F1 counts an entity as wrong when one character is off, which is harsher than a lawyer would be, while micro-F1@7 forces seven answers even when a case cites one. For a firm, this means every output should be treated as a draft with a confidence attached, never as a final answer.

7.3 Limitations and what you would do next

We trained on one shared GPU, so most runs have a single seed, and our validation sets are small (118 queries, 300 summaries), which makes differences below about 0.015 noise. The Phase-1 models never saw more than 1,024 tokens, yet Phases 4–7 run them on documents several times longer. We had no human legal evaluation, and two of our best systems (Phases 5 and 7) lean on lexical models. With more time we would continue pre-training the cleaned A4 model at 4,096 tokens, resolve party roles at the document level for NER, and pre-train the retriever on citation contexts mined from the Phase-1 corpus itself.

7.4 Surprising findings

Three results went against our expectations. The cleaned corpus lowered perplexity everywhere, yet NER got slightly worse with the new backbone (0.488 to 0.467). In Phase 7, the largest gains came from the structure of the data (one query per citation, normalising scores across the pool), not from the neural model. And local and leaderboard rankings did not always agree: in Phase 3 a single model beat our ensembles on Codabench.

8 Conclusion

We delivered what the brief asked for: four legal language models trained from scratch and six task systems built on them, all running on in-house hardware with no outside data or services. What moved the metrics was mostly how we handled the documents (cleaning the corpus, reading the end of a judgment, feeding longer sources, retrieving per citation) and combining models, while the choice among architectures and adaptation methods mattered less, with LoRMA and partial fine-tuning close to full fine-tuning. The main lesson is that a small model gains more from understanding the shape of legal text than from extra parameters. A sceptical partner should still not trust the summaries without reading the judgment, should expect the retriever to miss about half of the cited cases, and should know that NER misses roughly a third of entities.

LLM Usage Declaration

Completing this section is mandatory. Inconsistent answers relative to the code and the prose will be treated as an integrity issue.

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor (e.g., none / ChatGPT-4.x / Gemini x / Claude x / Grok / Copilot / other).

Answer: Claude (Anthropic), Claude Opus models, used through the Claude Code assistant in the terminal for all phases and for this report. No other LLM was used.

Q2. For each row, mark Yes or No and state the purpose in one short phrase. "Yes" is acceptable only for the uses permitted by the capstone document.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Explaining techniques (LoRMA, QLoRA, AdaLoRA, MBR, BM25, InfoNCE) and the task specifications.
Ideation / brainstorming at a high level	Yes	Proposing approaches per phase (chunking, citation windows, long sources, stacking).
Debugging small ancillary snippets	Yes	Fixing errors, out-of-memory failures and LaTeX build problems.
Plotting / visualization code	Yes	All `make_figures.py` scripts, figures and tables of the reports.
Grammar / writing style of the report	Yes	Wording of the phase reports and of this report.
Report structure / section wording	Yes	Structuring the reports to the template; drafting the methods and results text.
Designing the model architecture	Yes	Proposing the Phase-1 architectures and the task models of Phases 2–7.
Hyperparameter search / tuning	Yes	Choosing and running hyperparameter and decoding variants.
Analysis of results / conclusions	Yes	Drafting Sections 6–8 ("which system won", error analysis, conclusion).
Writing the core training / inference code	Yes	Most of the code in `legal_lm/` and `p2/`–`p7/`.
Other (specify)	Yes	Data audits, submission checks, git operations and the README.

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them (papers, blogs, GitHub repositories, textbooks, official starter notebook, evaluation scripts). These must also appear in the bibliography when they influenced the method.

Answer: Yes. The capstone document and its official starter notebook and evaluation scripts; the IL-TUR benchmark paper; the papers in the bibliography (LoRA, LoRMA, QLoRA, AdaLoRA, IA^3, adapters, prefix and prompt tuning, BitFit, ULMFiT, knowledge distillation, RoPE, ALiBi, GQA, Longformer, BPE, CRF, hierarchical attention, position interpolation, fusion-in-decoder,

MBR decoding, BM25, InfoNCE, LightGBM, BERTScore, ROUGE, BLEU); and the documentation of the open-source libraries used: PyTorch, Hugging Face `tokenizers` and `datasets`, `bert_score`, `rouge_score`, scikit-learn and LightGBM.

Q4. Apart from any permitted LLM use, what was *your* contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments, and analysis. Name at least two things you tried that failed and what you concluded from them.

[Draft from what the assistant observed; rewrite in your own words.] I set the goals for every phase, decided which ideas to pursue or drop, and approved the hybrid approaches of Phases 5 and 7. I ran and monitored the experiments on the shared GPU, uploaded every submission to Codabench, tracked the leaderboard scores and ranks, and used them to decide where to push further. I checked the data issues that changed the protocol, such as the labelled test file and the test–dev overlap in Phase 4. ... Two things that failed: the cross-encoder re-ranker in Phase 7 memorised a single batch but did not generalise from 817 training queries, which suggested that it needs far more training pairs than we had; and token-level ensembling in Phase 6 scored below the best single model, so we combined complete summaries with consensus selection instead. ...

GitHub repository: https://github.com/manishkr21/capstone_llm

References

[1] A. Joshi, S. Paul, A. Sharma, P. Goyal, S. Ghosh, and A. Modi, "IL-TUR: Benchmark for Indian legal text understanding and reasoning," *arXiv preprint arXiv:2407.05399*, 2024.

[2] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of ACL*, 2016.

[3] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "CTRL: A conditional transformer language model for controllable generation," *arXiv preprint arXiv:1909.05858*, 2019.

[4] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI technical report*, 2019.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.

[6] O. Press, N. A. Smith, and M. Lewis, "Train short, test long: Attention with linear biases enables input length extrapolation," in *International Conference on Learning Representations*, 2022.

[7] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[8] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of EMNLP*, 2023.

[9] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020.

[10] J. Howard and S. Ruder, "Universal language model fine-tuning for text classification," in *Proceedings of ACL*, 2018.

[11] E. Ben Zaken, Y. Goldberg, and S. Ravfogel, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proceedings of ACL*, 2022.

[12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[13] H. Bihany, S. Patel, and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," in *Findings of ACL*, 2025.

[14] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, 2023.

[15] Q. Zhang, M. Chen, A. Bukharin, P. He, Y. Cheng, W. Chen, and T. Zhao, "AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning," in *International Conference on Learning Representations*, 2023.

[16] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *International Conference on Machine Learning*, 2019.

[17] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Advances in Neural Information Processing Systems*, 2022.

[18] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proceedings of ACL*, 2021.

[19] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proceedings of EMNLP*, 2021.

[20] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[21] J. Lafferty, A. McCallum, and F. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in *International Conference on Machine Learning*, 2001.

[22] Z. Yang, D. Yang, C. Dyer, X. He, A. Smola, and E. Hovy, "Hierarchical attention networks for document classification," in *Proceedings of NAACL*, 2016.

[23] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, 2017.

[24] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proceedings of EACL*, 2021.

[25] S. Chen, S. Wong, L. Chen, and Y. Tian, "Extending context window of large language models via positional interpolation," *arXiv preprint arXiv:2306.15595*, 2023.

[26] S. Kumar and W. Byrne, "Minimum Bayes-risk decoding for statistical machine translation," in *Proceedings of HLT-NAACL*, 2004.

[27] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," in *Foundations and Trends in Information Retrieval*, 2009.

[28] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

A Method details

This appendix documents every phase in more detail than the page limit allows: shared components (subsection A.1), then for each phase the pipeline, the input processing, the model, training and selection, inference, all experiments with their local scores (tables generated from the run outputs by the phase reports), and the six best finished Codabench submissions with the settings that distinguish them. "Local" scores: P1 dev BERTScore F1 (200 leak-free prompts); P2 strict macro-F1 (10 held-out documents); P3 out-of-fold macro-F1 (5 folds); P4 clean-validation macro-F1 at the tuned threshold (stacks: nested split-half); P5 cross-fitted macro-F1; P6 BERTScore F1 (300 validation documents; beam 4: 100); P7 dev micro-F1@7.

A.1 Shared components

Backbone wrapper. One module turns each Phase-1 model into an encoder that returns a hidden state per input token: A1 and A4 run their blocks with a causal (optionally bidirectional) mask, A3 runs with the whole input as its bidirectional prefix, and A2 runs its decoder over the input while cross-attending to its encoder (the Phase-1 A2 encoder output is nearly constant across inputs). The language-model head is dropped.

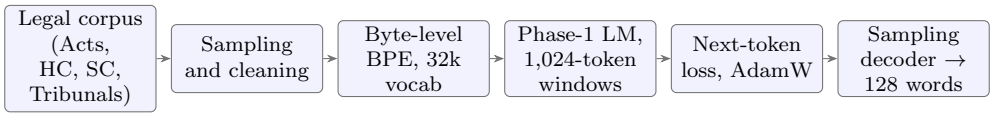
Adaptation techniques. For a frozen pretrained weight $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and input x:

- **Full fine-tuning**: every weight is trained. **Partial**: only the top k blocks and the final norm. **Gradual unfreezing**: the top block first, one more block every 300 steps. **BitFit**: biases and norm gains only.
- **LoRA**: $h = W_0 x + \frac{\alpha}{r} BAx$, $A \in \mathbb{R}^{r \times d_{\text{in}}}$, $B \in \mathbb{R}^{d_{\text{out}} \times r}$, $B = 0$ at start. **rsLoRA**: scale $\alpha/\sqrt{r}$.
- **LoRMA**: multiplicative update $h = (I + \frac{\alpha}{r} BA)\, W_0 x$, so the low-rank term transforms the output of the pretrained map instead of adding to it.
- **QLoRA**: W_0 is stored in 4-bit NormalFloat with one scale per block of 64 weights and dequantised on the fly; LoRA trains on top.
- **AdaLoRA**: SVD-form update $P\Lambda Q$ with an orthogonality penalty; singular values are pruned by an importance score from rank 12–32 to a smaller budget during training.
- **IA^3**: learned vectors rescale keys, values and the MLP's hidden activations, $k \odot l_k$, $v \odot l_v$, $\text{MLP}_{\text{in}} \odot l_{\text{ff}}$ (0.2–0.3% of parameters).
- **Houlsby adapters**: $x + W_{\text{up}}\, \sigma(W_{\text{down}} x)$ after attention and after the MLP (bottleneck 64–128), plus layer norms.
- **Prefix tuning**: P learned key/value vectors per layer, produced by a small MLP from P embeddings and prepended to every attention. **Prompt tuning**: P learned input embeddings replace the first P positions.

Knowledge distillation. The student keeps a subset of the teacher's blocks (Phase-1 weights) and is fully fine-tuned with $\mathcal{L} = \alpha\, \mathcal{L}_{\text{task}} + (1 - \alpha)\, T^2\, \text{KL}\big(\sigma(z_t/T) \,\|\, \sigma(z_s/T)\big)$; the teacher logits z_t come from one run or the mean of several runs, computed once on the training data.

Sequence and ranking objectives. **CRF** (P3): $p(y \mid x) \propto \exp\big(\sum_i e_{i,y_i} + \sum_i T_{y_{i-1},y_i}\big)$ with emissions e and transition matrix T, decoded by Viterbi. **InfoNCE** (P7): for query q with gold set G and batch candidates C, $\mathcal{L} = -\frac{1}{|G|} \sum_{g \in G} \log \frac{e^{s(q,g)/\tau}}{e^{s(q,g)/\tau} + \sum_{n \in C \setminus G} e^{s(q,n)/\tau}}$. **MBR selection** (P6): among candidates $c_1..c_k$, pick $\arg\max_i \sum_{j \neq i} \big(\text{R1}(c_i, c_j) + \text{R2}(c_i, c_j)\big)$.

A.2 Phase 1 — Legal language model pre-training



Data. R0 used the starter sample (about 200M tokens). The cleaned corpus samples whole files from the four sources, normalises characters and markup, repairs OCR word stutter (2.34M stuttered words), drops documents that are too short (379,813) or have few letters (29,393), removes 106,717 exact and 204,246 MinHash near-duplicates, and builds a validation split sharing no text with either corpus (764 leaking validation documents dropped); it yields 1.006B training tokens.

Tokenizer. Byte-level BPE with 32,000 tokens trained on 100,000 training documents (1.25 tokens per word); a 512-word prompt is about 640 tokens.

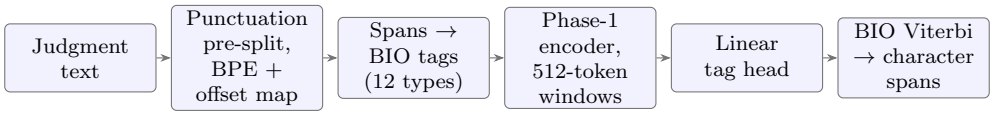
Model and training. Four architectures (Table 4); next-token cross-entropy (A2: continuation of a random-length prefix; A3: a random prefix read bidirectionally, half the examples fully causal); AdamW $\beta = (0.9, 0.95)$, weight decay 0.1, peak LR 6×10^{-4}, 200 warm-up steps, cosine decay to 6×10^{-5}; 8 sequences $\times$ 8 accumulation $\times$ 1,024 tokens per step; R0/R1 3,000 steps, R2 12,260 more; validation every 250 steps on the 257 tokens after a 768-token prefix.

Inference. Prompt pre-filled into a KV cache; temperature 0.8, top-k 50, top-p 0.95, repetition penalty 1.1 over the last 256 tokens; end-of-text banned until 128 words.

		Perplexity			Dev generation		
Model	Run	Old val	Leak-free	Fair	BERTScore	ROUGE-L	BLEU-4
A1 GPT	R0	11.36	17.49	12.93	0.8286	0.1920	0.0582
	R1	–	10.47	11.82	0.8292	0.1988	0.0736
	R2	–	7.23	8.37	0.8359	0.2137	0.0938
A2 enc-dec	R0	21.79	38.32	25.26	0.8170	0.1725	0.0275
	R1	–	20.78	22.31	0.8162	0.1710	0.0291
	R2	–	11.75	13.12	0.8245	0.1844	0.0398
A3 prefix-LM	R0	12.11	19.46	13.89	0.8255	0.1851	0.0473
	R1	–	10.95	12.42	0.8277	0.1941	0.0596
	R2	–	7.76	8.98	0.8341	0.2092	0.0877
A4 efficient	R0	11.53	18.75	13.32	0.8269	0.1861	0.0502
	R1	–	9.91	11.25	0.8325	0.2045	0.0796
	R2	–	7.17	8.26	0.8358	0.2127	0.0928

#	Submission	Codabench	Local	Method (key settings)
1	submission_2026-09-26_22-29-00	0.8286	0.8286	**A1 GPT** (R0): decoder-only, learned positions, pre-LayerNorm, GELU; 8 layers, $d = 768$, 12 heads; 82.1M.
2	submission_2026-09-27_01-22-10	0.8285	0.8255	**A3 prefix-LM** (R0): bidirectional prefix, causal continuation, ALiBi positions; 8 layers, $d = 768$; 81.3M.
3	submission_2026-09-27_01-29-15	0.8280	0.8269	**A4 efficient** (R0): RoPE, RMSNorm, SwiGLU, grouped-query attention (12 q / 4 kv), 256-token sliding window, global layer every 3rd; 10 layers; 87.5M.
4	submission_2026-09-26_22-38-32	0.8184	0.8170	**A2 encoder–decoder** (R0): 6+6 layers, sinusoidal positions, $d = 640$; random prefix/continuation splits; 89.4M.
5	submission.zip (26 Sep)	0.8006	–	Early upload before the four-architecture runs; the file does not identify the model.
6	submission.zip (25 Sep)	0.8002	–	Early upload, as above.

A.3 Phase 2 — Legal named-entity recognition



Input. Runs of punctuation are split before BPE so entity boundaries fall on token boundaries; the tokenizer's offset table maps every token to its characters, so a gold span becomes B-/I- tags on its tokens and a predicted tag sequence maps back to exact character offsets (round trip: 99.45% of gold spans).

Model. Phase-1 encoder (R0 checkpoints) → dropout 0.1 → linear layer over $2 \times 12 + 1 = 25$ BIO tags.

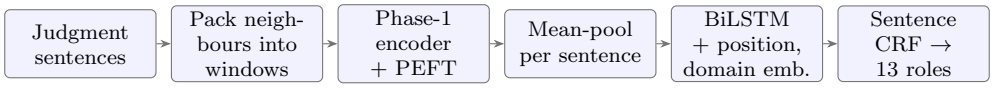
Training and selection. Windows of 512 tokens with stride 256; 16 windows per step; AdamW, 10% warm-up, up to 15 epochs with patience 5; head learning rate 10^{-3}; best epoch by strict span macro-F1 on 10 of the 70 training documents (later confirmed on three different 10-document splits).

Inference. Overlapping windows (stride 128), each token keeps the logits of the window in which it has the most context; Viterbi decoding restricted to valid BIO transitions (I-X only after B-X or I-X).

Run	Arch	Method	Trainable	%	Macro-F1	P	R	Micro-F1	Span	Min	GB	MB
E1_a3_full	A3	full	81.30M	100.0	0.4267	0.447	0.441	0.507	0.560	2.1	3.7	325.2
E2_a4_lorma	A4	lorma	2.23M	2.5	0.4857	0.517	0.478	0.510	0.573	4.4	4.5	9.0
E3_a1_lora	A1	lora	1.79M	2.1	0.3564	0.395	0.356	0.451	0.503	2.8	3.2	7.2
E4_a2_adapter	A2	adapter	2.04M	2.2	0.3892	0.422	0.389	0.447	0.489	3.1	2.9	8.2
E5_a4_partial	A4	partial	25.19M	28.8	0.4697	0.488	0.474	0.520	0.559	1.4	2.3	100.8
E6_kd_a3_small	A3	full (KD)	52.95M	100.0	0.3443	0.372	0.344	0.434	0.483	1.4	2.6	211.8
diag_E4_a2_adapter_encoder	A2 (enc.)	adapter	1.02M	2.0	0.1116	0.155	0.094	0.186	0.186	1.5	1.4	4.1
diag_a2_partial_encoder	A2 (enc.)	partial	14.79M	29.6	0.1277	0.171	0.113	0.201	0.211	1.0	1.1	59.2
diag_a2_full	A2 (enc.)	full	50.04M	100.0	0.1493	0.171	0.147	0.226	0.265	0.7	2.0	200.2

#	Submission	Codabench	Local	Method (key settings)
1	E2_a4_lorma	0.4970	0.4857	**A4 + LoRMA**: $r = 16$, $\alpha = 32$, on q, k, v, o, gate, up, proj; bidirectional attention; LR 10^{-3}; 2.2M trained.
2	E1_a3_full	0.4815	0.4267	**A3 full fine-tuning**: every weight; LR 10^{-4}; 81.3M trained.
3	E5_a4_partial	0.4753	0.4697	**A4 partial**: top 4 of 10 blocks + final norm trainable; bidirectional; LR 2×10^{-4}.
4	E3_a1_lora	0.4239	0.3564	**A1 + LoRA**: $r = 16$, $\alpha = 32$ on q, k, v, o, fc, proj; bidirectional; LR 10^{-3}; 1.8M trained.
5	E4_a2_adapter	0.4015	0.3892	**A2 + Houlsby adapters**: bottleneck 64 after attention and MLP; decoder states; LR 10^{-3}.
6	E6_kd_a3_small	0.3799	0.3443	**KD**: A3 student with blocks 1, 3, 5, 7 (52.9M), full fine-tuning; teacher E1; $\alpha = 0.5$, $T = 2$.

A.4 Phase 3 — Rhetorical role prediction



Input. Each sentence is cut to at most 128 tokens; consecutive sentences are packed into backbone windows of 512 (or 1,024) tokens, so every sentence is encoded together with its neighbours.

Model. Mean of the backbone states over each sentence's tokens → concatenated with a relative-position bucket (20 buckets) and a domain embedding (CL / IT) → BiLSTM (hidden 256) over the document's sentences → emission scores for 13 roles → linear-chain CRF.

Loss. CRF negative log-likelihood with logit adjustment: emissions are shifted by $\tau \log \pi_y$ (π = training role frequencies) during training so that rare roles are not drowned out.

Training and selection. Chunks of 128 consecutive sentences, 4 chunks per step; backbone (PEFT) and head learning rate 10^{-3}; 12 epochs, patience 4 (tuned runs: 20, patience 6, $\tau = 0.5$, causal mask); 5-fold document-level cross-validation stratified by domain; out-of-fold macro-F1 is the selection metric.

Inference and ensembles. Test emissions are averaged over the 5 fold models (ensembles: over several runs' fold models), then Viterbi. Optional class-bias calibration fitted on out-of-fold emissions.

Run	Arch	Method	Trainable	OOF F1	CL	IT	Fold mean	P	R	Min	GB	MB
E1_a4_lorma	A4	lorma	2.81M	0.455	0.462	0.468	0.478±0.028	0.438	0.509	9.3	4.6	11.3
E1_a4_lorma_origP1	A4	lorma	2.81M	0.427	0.434	0.403	0.437±0.021	0.412	0.495	9.5	4.6	11.3
E0_a4_lorma_sentonly	A4	lorma	2.42M	0.361	0.324	0.378	0.360±0.019	0.340	0.412	25.5	9.1	9.7
E2_a1_lora	A1	lora	2.37M	0.330	0.347	0.310	0.335±0.033	0.324	0.377	6.0	3.2	9.5
E3_a3_rslora	A3	rslora	7.68M	0.441	0.458	0.461	0.473±0.025	0.433	0.493	7.3	3.5	30.7
E4_a2_ia3	A2	ia3	0.62M	0.406	0.417	0.417	0.415±0.029	0.397	0.482	7.1	3.2	2.5
E5_a4_qlora	A4	qlora	2.77M	0.423	0.439	0.424	0.458±0.018	0.405	0.485	9.8	4.3	11.1
E6_kd_a4_small	A4	full (KD)	62.94M	0.403	0.412	0.385	0.434±0.026	0.392	0.449	14.4	4.1	251.8
E7_a3_adalora	A3	adalora	1.93M	0.396	0.414	0.393	0.416±0.030	0.379	0.476	8.5	3.4	7.8
E8_a4_prefix	A4	prefix	3.63M	0.326	0.354	0.285	0.331±0.028	0.322	0.409	7.5	2.9	14.5
E9_a1_prompt	A1	prompt	0.62M	0.272	0.303	0.236	0.287±0.035	0.277	0.353	4.9	2.0	2.5

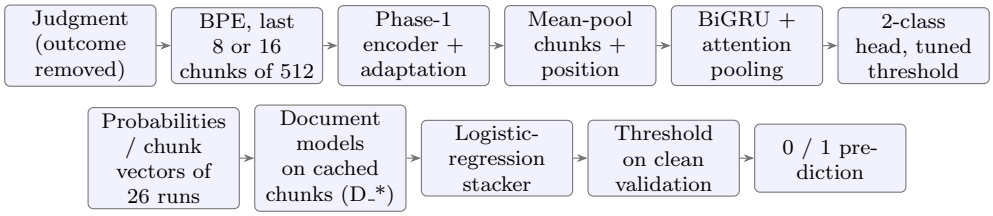
Table 9: Phase-3 tuning runs (parent, purpose, change, out-of-fold macro-F1).

Run	Parent	Purpose	Change	OOF F1	Δ	Fold mean
T1	E1	E1 with 512-sentence training chunks (the BiLSTM sees most of a document; targets Dissent, rulings)	chunk_sents=512, chunks_per_step=1	0.416	-0.038	0.432±0.049
T2	E1	E1 with weaker logit adjustment (macro recall 0.51 ¿ precision 0.44 at tau 1)	logit_adjust_tau=0.5	0.459	+0.004	0.509±0.033
T3	E3	E3 trained longer (best epoch was 12, 11, 12, 9 of 12)	epochs=20, patience=6	0.467	+0.026	0.475±0.022
T4	E2	E2 with causal attention (A1 was pre-trained causally only)	attention=causal	0.477	+0.147	0.499±0.010
T6	E1	E1 with causal attention only (isolates the attention effect on A4)	attention=causal	0.459	+0.004	0.489±0.017
T7	E1	E1 (A4 LoRMA) causal + 20 epochs + tau 0.5	attention=causal, epochs=20, logit_adjust_tau=0.5, patience=6	0.500	+0.045	0.547±0.023
T8	E2	T4 (A1 LoRA causal) + 20 epochs + tau 0.5 (T4 peaked at its last epoch in 3/5 folds)	attention=causal, epochs=20, logit_adjust_tau=0.5, patience=6	0.491	+0.161	0.538±0.034
T9	E3	T3 (A3 rsLoRA, 20 epochs) + tau 0.5	epochs=20, logit_adjust_tau=0.5, patience=6	0.466	+0.025	0.519±0.026
T10	E5	E5 (A4 QLoRA) causal + 20 epochs + tau 0.5	attention=causal, epochs=20, logit_adjust_tau=0.5, patience=6	0.485	+0.062	0.544±0.030
T11	E8	E8 (A4 prefix tuning) causal + 20 epochs + tau 0.5	attention=causal, epochs=20, logit_adjust_tau=0.5, patience=6	0.447	+0.121	0.504±0.035

Run	Parent	Purpose	Change	OOF F1	Δ	Fold mean
T12	E9	E9 (A1 prompt tuning) causal + 20 epochs + tau 0.5	attention=causal, epochs=20, logit_adjust_tau=0.5, patience=6	0.422	+0.149	0.456±0.033
T13	E6	E6 (KD, A4 6/10-layer student, full FT) with the stronger T7 teacher, causal, 20 epochs, tau 0.5	attention=causal, epochs=20, logit_adjust_tau=0.5, patience=6, teacher=T7	0.498	+0.095	0.537±0.026
T14	T8	T8 with 1024-token windows (A1 was pretrained on 1024-token blocks)	max_windows=16, window=1024, max_windows=8, window=1024	0.501	+0.009	0.558±0.028
T15	T8	T8 with LoRA rank 32 (alpha 64, same scale)	alpha=64, r=32	0.493	+0.001	0.548±0.021
T16	T8	T8 with half the learning rate (fold 3 peaked at epoch 3)	lr=0.0005	0.497	+0.005	0.552±0.022
T17	T8	T8 with a wider document layer (BiLSTM hidden 512)	hidden=512	0.500	+0.008	0.557±0.025
T18	T8	T8, another seed (same folds) – for a seed ensemble	seed=2024	0.500	+0.008	0.548±0.022
T19	T8	T8, a third seed (same folds) – for a seed ensemble	seed=7	0.500	+0.008	0.553±0.026

#	Submission	Codabench	Local	Method (key settings)
1	T14_t8_win1024	0.5750	0.5007	**A1 + LoRA** ($r = 16$, $\alpha = 32$), tuned, **1,024-token windows** (the pre-training block length); 2.4M trained.
2	T15_t8_r32	0.5736	0.4926	As T8 with LoRA rank 32 ($\alpha = 64$); 4.1M trained.
3	C6_ens_8_nocal	0.5711	0.5200	**Ensemble of 8**: T14, T17 (BiLSTM 512), T18/T19 (seeds), T7 (A4 LoRMA), T13 (KD 6-block A4), T10 (A4 QLoRA), T9 (A3 rsLoRA); no calibration.
4	T8_e2_causal_ep20_tau05	0.5709	0.4915	**A1 + LoRA**, tuned, 512-token windows.
5	C7_ens_fold_plus_all	0.5707	0.5211	**C7 ensemble** (as C6 with lexical cue flags in T20/T22): fold models plus their all-data retrains averaged.
6	C5_ens_a1x4_nocal	0.5696	0.5179	**Ensemble of 4 A1 LoRA taggers**: T14, T17, T18, T19.

A.5 Phase 4 — Court judgment prediction



Input. The judgment is tokenized once; 512-token chunks are counted from the end and the last 8 (V1/V2: 16) are kept; ablations keep the first 8, first 4 + last 4, or 8 evenly spread.

Model. Chunk vector c_j = mean of the backbone states → projection to 256 + position embedding → BiGRU → attention pooling $\sum_j a_j h_j$, $a = \mathrm{softmax}(w^\top h_j)$ → 2 logits.

Training and selection. Class-weighted cross-entropy (weights inverse to the 59/41 training frequencies); 16 documents per step; 2 epochs; 6% warm-up and cosine decay; validation every 250 steps on the 1,988 development documents not in the test set; the decision threshold on $P(y{=}1)$ is tuned on the same documents.

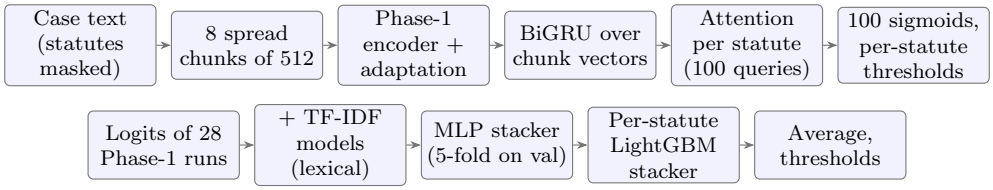
Document models and stacking. `embed_chunks.py` stores every chunk vector of fine-tuned encoders once; `doc_train.py` trains light document layers (mean, attention, GRU, LSTM, 2-layer

transformer) on the concatenated vectors of 1, 3 or 5 encoders. `stack_submission.py` fits an ℓ_2-regularised logistic regression on clean validation over every run's logit and reports a nested split-half estimate.

Run	Arch	Method	Chunks	Trainable	%	F1	F1@0.5	thr	P	R
E1_a1_full	A1	full fine-tuning	last	82.58M	100.0	0.7698	0.7398	0.42	0.775	0.771
E2_a4_lorma	A4	LoRMA+	last	2.72M	3.0	0.7535	0.7325	0.35	0.754	0.754
E3_a3_adapter	A3	Houlsby adapters	last	2.12M	2.5	0.7541	0.7211	0.41	0.756	0.755
E4_a2_bitfit	A2	BitFit	last	0.60M	0.7	0.6614	0.5330	0.29	0.663	0.662
E5_a1_adalora	A1	AdaLoRA	last	1.84M	2.2	0.6001	0.4368	0.40	0.603	0.602
E6_a4_partial	A4	partial (top 4 blocks)	last	25.68M	29.2	0.7854	0.7502	0.39	0.787	0.786
E7_a3_gradual	A3	gradual unfreezing	last	81.79M	100.0	0.7726	0.7228	0.31	0.773	0.773
E8_kd_a4_small	A4	KD student (full FT)	last	62.85M	100.0	0.7756	0.7666	0.56	0.776	0.776
A_e6_first_last	A4	partial (top 4 blocks)	first_last	25.68M	29.2	0.7785	0.7717	0.47	0.780	0.779
A_e6_spread	A4	partial (top 4 blocks)	spread	25.68M	29.2	0.7776	0.7541	0.43	0.778	0.778
A_e6_first	A4	partial (top 4 blocks)	first	25.68M	29.2	0.7535	0.6986	0.34	0.754	0.754

#	Submission	Codabench	Local	Method (key settings)
1	STACK_with_fusion	0.8121	0.8225	**Stacker over 26 runs**: E1–E8, 3 chunk-selection ablations, V1–V6, 9 D_* document models (fusing 1, 3 or 5 encoders); threshold 0.52.
2	STACK_all	0.8048	0.8160	**Stacker over 17 end-to-end runs** (no document models); threshold 0.50.
3	ENS_greedy5	0.8009	0.8238	**Mean of 5**, greedy selection: V1 (A4 partial, 16 chunks), A4 on first chunks, E7 (A3 gradual unfreezing), E4 (A2 BitFit), V6 (6-block A4 KD from a 3-model ensemble).
4	ENS_8_views_kd	0.7963	0.8068	**Mean of 8**: E6, E7, E1, E3, E2, first+last and spread chunk views, E8 KD student; threshold 0.42.
5	ENS_greedy4	0.7955	0.8219	As ENS_greedy5 without V6; threshold 0.36.
6	ENS_top5	0.7930	0.7974	**Mean of 5 best first-round runs**: E6 (A4 partial), E7 (A3 gradual), E1 (A1 full), E3 (A3 adapters), E2 (A4 LoRMA+); threshold 0.39.

A.6 Phase 5 — Legal statute identification



Input. Eight 512-token chunks spread evenly over the document (the whole document when it has at most 8 chunks, 88% of validation documents); variant V1 centres chunks on the masked `<SECTION>`/`<ACT>` placeholders.

Model. Chunk vectors → projection to 384 + position embedding → BiGRU states H → for each statute ℓ: $a_\ell = \mathrm{softmax}(Hq_\ell)$, $z_\ell = w_\ell^\top (a_\ell^\top H) + b_\ell$, with a query q_ℓ and output vector w_ℓ per statute.

Loss. $\sum_\ell \mathrm{BCE}(z_\ell, y_\ell)$ with positive weight $\min(30, \sqrt{n_\ell^- / n_\ell^+})$.

Training and selection. 16 documents per step, 3 epochs (later 5–8), learning rates per technique (5×10^{-5} full / KD to 3×10^{-3} prompt), head 10^{-3}; a fixed 2,000-document validation subset scored every 800–2,000 steps.

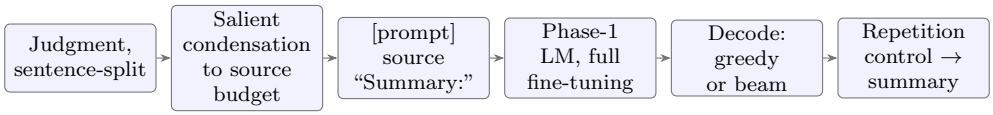
Thresholds. Per statute, a grid search on validation (statutes with fewer than 5 validation positives use the global threshold); at least one statute per document. Reported scores are cross-fitted (thresholds fitted on one half, scored on the other, both directions).

Stacking. Out-of-fold over the validation split (5 folds), 5 fold models averaged for test. MLP: input = every run's 100 logits, one hidden layer of 512, dropout 0.3, 30 epochs. LightGBM: per statute, features = each run's logit for it, the mean logit of all statutes, and the across-run spread.

Run	Arch	Method	Ep.	Trainable	%	Global	CF	P	R	Micro	Subset
E1	A4	LoRMA	3	3.27M	3.6	0.3692	0.3771	0.356	0.421	0.477	0.039
E2	A1	full fine-tuning	3	83.13M	100.0	0.3501	0.3606	0.337	0.412	0.462	0.035
E3	A3	Houlsby adapters	3	2.67M	3.2	0.3569	0.3642	0.342	0.412	0.464	0.033
E4	A2	partial (top 4 blocks)	3	27.29M	30.2	0.3301	0.3391	0.308	0.404	0.439	0.025
E5	A4	partial (top 4 blocks)	3	26.24M	29.6	0.3582	0.3700	0.349	0.423	0.467	0.040
E6	A1	QLoRA (NF4)	3	2.83M	10.0	0.3619	0.3705	0.350	0.420	0.468	0.034
E7	A3	AdaLoRA	3	2.39M	2.9	0.3403	0.3533	0.335	0.402	0.455	0.029
E8	A4	prefix tuning	3	4.10M	4.5	0.3474	0.3526	0.327	0.404	0.452	0.030
E9	A1	prompt tuning	3	1.08M	1.3	0.3372	0.3453	0.320	0.399	0.448	0.031
E10	A4	KD student (6 blocks, TEACHER)	3	63.40M	100.0	0.3744	0.3816	0.362	0.425	0.479	0.040
V1	A4	partial (top 4 blocks)	3	26.24M	29.6	0.3601	0.3705	0.350	0.422	0.464	0.033
V2	A4	partial (top 4 blocks)	5	26.24M	29.6	0.3769	0.3813	0.364	0.422	0.476	0.043
V3	A1	full fine-tuning	5	83.13M	100.0	0.3673	0.3715	0.355	0.414	0.471	0.043
W1	A4	partial (top 4 blocks)	8	26.24M	29.6	0.3822	0.3884	0.382	0.413	0.489	0.053
W2	A4	full fine-tuning	5	88.57M	100.0	0.3768	0.3853	0.375	0.418	0.486	0.047
W3	A3	Houlsby adapters	5	2.67M	3.2	0.3693	0.3763	0.357	0.417	0.477	0.039
W4	A4	LoRMA	5	3.27M	3.6	0.3842	0.3882	0.371	0.425	0.487	0.044
W5	A1	QLoRA (NF4)	5	2.83M	10.0	0.3815	0.3879	0.372	0.429	0.482	0.041
W6	A4	KD student (6 blocks, TEACHER2)	6	63.40M	100.0	0.3931	0.3971	0.383	0.432	0.491	0.050
X1	A1	QLoRA (NF4)	8	2.83M	10.0	0.3833	0.3868	0.374	0.419	0.485	0.043
X2	A4	full fine-tuning	8	88.57M	100.0	0.3810	0.3813	0.374	0.405	0.483	0.049
X3	A4	KD student (6 blocks, TEACHER3)	8	63.40M	100.0	0.4007	0.4037	0.395	0.428	0.502	0.056
Y1	A4	KD student (6 blocks, TEACHER4)	8	63.40M	100.0	0.4009	0.4018	0.400	0.421	0.503	0.055
Y2	A4	KD student (6 blocks, TEACHER4)	8	63.40M	100.0	0.4076	0.4107	0.402	0.440	0.508	0.054
Y3	A1	KD student (6 blocks, TEACHER4)	8	68.95M	100.0	0.3963	0.3974	0.384	0.429	0.498	0.049
Y4	A3	full fine-tuning	5	82.34M	100.0	0.3535	0.3617	0.338	0.418	0.454	0.033
Y5	A2	full fine-tuning	5	90.43M	100.0	0.3473	0.3524	0.321	0.415	0.450	0.031

#	Submission	Codabench	Local	Method (key settings)
1	STACK11_full	0.4624	0.4603	MLP (2 seeds) + LightGBM (3 seeds) over 28 Phase-1 runs and 6 lexical models, incl. a TF-IDF model trained on train + validation folds.
2	STACK10_seedavg	0.4587	0.4602	As STACK9, averaged over independently seeded stackers.
3	STACK9_gbm_mlp	0.4579	0.4595	LightGBM stacker (32 inputs) averaged with the STACK8 MLP stacker.
4	STACK8_hybrid	0.4493	0.4473	MLP stacker, 27 Phase-1 runs + 4 lexical (word 1–2 / 1–3-gram, character TF-IDF, linear SVM).
5	STACK7_hybrid	0.4457	0.4436	MLP stacker, 27 Phase-1 runs + word TF-IDF logistic regression.
6	STACK6_all	0.4231	0.4233	MLP stacker over 27 Phase-1 runs only: all four architectures, 8 PEFT methods, 6 KD students.

A.7 Phase 6 — Abstractive summarization



Input. The document is sentence-split and tokenized once. If it exceeds the source budget B (1,280–8,192 tokens), *salient* condensation keeps the opening of the case ($0.15B$; S7c $0.3B$) and fills the rest with the sentences closest (cosine) to the document's own TF-IDF centroid, restored to document order; *spread* keeps evenly spaced sentences.

Model. One sequence [prompt][left-padded source][" `Summary:`"][summary][end]; the loss is the cross-entropy on summary tokens only (up to 768). A3 attends bidirectionally over prompt and

source. A2: the source is split into 4 chunks of 1,024 tokens, each encoded separately (fusion-in-decoder). Long sources: A1's learned position table $E \in \mathbb{R}^{1024\times d}$ is linearly interpolated to the needed length; A4 (RoPE) and A3 (ALiBi) are used as they are.

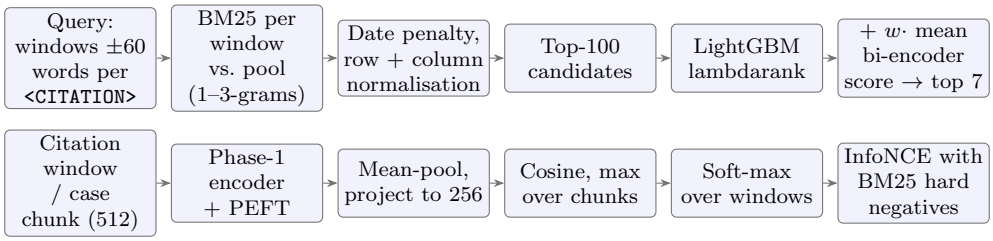
Training and selection. AdamW, 16 sequences per step (micro-batch × accumulation), warm-up 100 (50 when continuing a run), cosine decay, weight decay 0.01, dropout 0.1, bf16; 4–6 epochs from Phase 1, or 2–3 epochs at 4–5 × 10^{-5} when continuing a shorter-source run; best state by validation loss.

Inference. KV-cached greedy decoding (or beam search with cache re-ordering), 300–800 new tokens, repetition penalty 1.15, no repeated 3-grams. `mbr.py` selects per document the candidate with the highest ROUGE-1 + ROUGE-2 agreement with the other candidates; `ensemble.py` averages next-token log-probabilities of several models.

Run	Arch	Method	Source	Ep.	Trainable	Loss	BERTScore F1	ROUGE-1	ROUGE-L	Words
F1	A4	full fine-tuning	1,280	4	87.51M	2.056	0.8486	0.495	0.244	414
F2	A2	full fine-tuning + FiD	4×1024	4	89.42M	2.727	0.8329	0.421	0.203	371
E1	A4	LoRMA	1,280	4	4.42M	2.088	0.8480	0.499	0.245	424
E2	A1	LoRA	1,280	4	5.14M	2.133	0.8493	0.526	0.260	472
E3	A3	Houlsby adapters	1,280	4	3.19M	2.175	0.8481	0.508	0.244	439
E4	A2	QLoRA (NF4) + FiD	4×1024	4	5.41M	2.816	0.8315	0.407	0.199	349
E5	A3	prefix tuning	1,280	4	6.75M	2.336	0.8382	0.482	0.215	568
E6	A1	AdaLoRA	1,280	4	5.14M	2.205	0.8463	0.521	0.248	519
E7	A4	prompt tuning	1,280	4	0.05M	2.411	0.8336	0.458	0.207	582
L2	A1	full fine-tuning	1,280	6	82.88M	2.066	0.8505	0.524	0.267	455
L3	A3	full fine-tuning	1,280	6	81.28M	2.096	0.8484	0.501	0.246	417
K1	A4	KD student, 6 blocks (full fine-tuning)	1,280	6	62.33M	2.258	0.8479	0.503	0.246	418
S2	A1	LoRA	1,280	4	5.14M	2.145	0.8452	0.505	0.246	471
S1	A4	full fine-tuning	2,560	6	87.51M	1.752	0.8519	0.509	0.262	410
S3	A4	full fine-tuning	4,096	5	87.51M	1.593	0.8518	0.520	0.265	427
S3b	A4	full fine-tuning	4,096	3*	87.51M	1.593	0.8528	0.517	0.266	413
S4	A4	full fine-tuning	8,192	3*	87.51M	1.536	0.8515	0.517	0.267	419
S5	A3	full fine-tuning	4,096	5	81.28M	1.595	0.8523	0.510	0.262	409
S9	A3	full fine-tuning	6,144	2*	81.28M	1.476	0.8519	0.501	0.264	398
S6	A1	full fine-tuning	2,560	6	83.86M	1.732	0.8520	0.540	0.282	502
S7	A1	full fine-tuning	4,096	3*	85.04M	1.539	0.8544	0.513	0.273	432
S7b	A1	full fine-tuning	4,096	3*	85.04M	1.530	0.8543	0.505	0.273	402
S7c	A1	full fine-tuning	4,096	3*	85.04M	1.519	0.8554	0.518	0.280	427
S8	A1	full fine-tuning	8,192	2*	88.19M	1.453	0.8530	0.506	0.271	418

#	Submission	Codabench	Local	Method (key settings)
1	S3_a4_full_src4096_beam4	0.8558	0.8522	**A4 full**, 4,096-token source, 5 epochs from Phase-1; **beam search, 4 beams**.
2	S4_a4_full_src8192	0.8539	0.8515	**A4 full**, 8,192-token source, 3 epochs continued from S3 (LR 5×10^{-5}); greedy.
3	S1_a4_full_src2560	0.8538	0.8519	**A4 full**, 2,560-token source, 6 epochs; greedy.
4	S8_a1_full_src8192	0.8520	0.8530	**A1 full**, 8,192-token source (interpolated positions), 2 epochs continued from S7 (4,096); greedy.
5	S3_a4_full_src4096_long	0.8520	0.8515	S3 re-decoded with 450–900 new tokens (longer summaries).
6	S3_a4_full_src4096	0.8520	0.8518	S3, greedy decoding.

A.8 Phase 7 — Prior case retrieval



Lexical retrieval. BM25 ($k_1 = 1.2$, $b = 0.75$, word 1–3-grams, document frequency 2 to 50% of the pool) with one query per citation window w; S_w = z-score of the window's BM25 scores over the pool; query score $S(q, c) = \max_{w \in q} S_w(c)$. Date penalty: -2 when the case's judgment year is after the query's. Column normalisation: $Z = \mathrm{z}_{\mathrm{row}}(S)$, $S' = Z + \mathrm{z}_{\mathrm{col}}(Z)$, where $\mathrm{z}_{\mathrm{col}}$ standardises each candidate's scores over all queries of the split; the window-level variant applies the same row/column step to S_w over all windows before the maximum ($\gamma = 2$ on the column term).

Bi-encoder. Windows (at most 256 tokens) and candidate chunks (512 tokens; 4 random chunks per case in training, 32 evenly spread at evaluation) go through the same backbone, mean pooling and a 256-dimensional projection, ℓ_2-normalised. $s(w, c) = \max_j \cos(w, c_j)$; $s(q, c) = \tau_w \log \sum_{w \in q} e^{s(w,c)/\tau_w}$ with $\tau_w = 0.05$.

Training. 8 queries per step; for each, 2 gold cases and 3 hard negatives sampled from the top 30 non-gold cases of per-citation BM25; all queries scored against all cases of the batch; multi-positive InfoNCE ($\tau = 0.05$), the query's own case masked; 10 epochs; best epoch by dev micro-F1@7. KD: KL between teacher and student score distributions over the batch at temperature 2τ.

Re-ranking. LightGBM lambdarank (600 trees, 15 leaves, learning rate 0.03, 3 seeds) over each query's top 100 first-stage candidates; features: row and column z-scores and ranks of 7 lexical scores (11 with the window-level scores), date difference and violation, log case length, log number of windows, whether the case is a query; trained on train, refit on train + dev for test. Final score = z(ranker) + $w \cdot \overline{\mathrm{z}_{\mathrm{row}} + \mathrm{z}_{\mathrm{col}}}$(bi-encoders).

Run	Arch	Method	Total	Trainable	%	Neural F1	P	R	Fused F1
E1	A3	LoRMA	85.0M	3.74M	4.4	0.3791	0.413	0.350	0.4658
E2	A1	full fine-tuning	82.3M	82.26M	100.0	0.3891	0.424	0.360	0.4703
E3	A4	partial (top 4 blocks)	87.7M	25.37M	28.9	0.3835	0.418	0.355	0.4680
E4	A2	QLoRA (NF4)	26.2M	5.57M	21.3	0.3635	0.396	0.336	0.4636
E5	A3	IA^3	81.5M	0.23M	0.3	0.3669	0.400	0.339	0.4691
E6	A1	prefix tuning	89.0M	6.91M	7.8	0.3313	0.361	0.306	0.4658
E7	A4	prompt tuning	87.7M	0.21M	0.2	0.3357	0.366	0.310	0.4614
K1	A1	KD student, 4 blocks	53.9M	53.91M	100.0	0.3691	0.402	0.341	0.4625
K0	A1	4 blocks, no teacher	53.9M	53.91M	100.0	0.3535	0.385	0.327	0.4725
E2b	A1	full fine-tuning	82.3M	82.26M	100.0	0.3891	0.424	0.360	0.4703
E2c	A1	full fine-tuning	82.3M	82.26M	100.0	0.3802	0.414	0.351	0.4703
E2d	A1	full fine-tuning, 6 hard negatives	82.3M	82.26M	100.0	0.3747	0.408	0.346	0.4691
E1b	A3	LoRMA	85.0M	3.74M	4.4	0.3691	0.402	0.341	0.4680
E3b	A4	partial (top 4 blocks)	87.7M	25.37M	28.9	0.3691	0.402	0.341	0.4647

#	Submission	Codabench	Local	Method (key settings)
1	STK4_rc_blend4	0.4950	0.4914	Re-ranker with row + column features and first stage, **+ 0.5 × mean row + column score of 4 bi-encoders** (E1 A3 LoRMA, E2 A1 full, E3 A4 partial, E5 A3 IA^3).
2	STK2_lex_blend4	0.4773	0.4869	Re-ranker (row features only) + 0.3 × mean row score of the same 4 bi-encoders.
3	STK1_e2	0.4743	0.4725	Re-ranker with the out-of-fold E2 score (2-fold runs) as an extra feature.
4	STK0_lex	0.4725	0.4769	Lexical re-ranker only, no neural input.
5	K1_kd_a1_4l_fused	0.4649	0.4625	**KD**: A1 student with blocks 0, 2, 5, 7 (53.9M), batch-score KL to teacher E2 ($\alpha = 0.5$); fused with BM25.
6	E2_a1_full_fused	0.4643	0.4703	**A1 full fine-tuning** bi-encoder (bidirectional, LR 3×10^{-5}), fused with BM25.